



에이전트는 하니스를 배운다

외부 실행 하니스를 학습하는 에이전트 정책 — EvoHarness-RL 정독

- A. 문제 — 하니스는 풍부해지는데 사용 정책은 프롬프트와 휴리스틱에 남는다
- B. 설계 — Belief·Progress·Experience 세 상태와 track·commit·recall·note 네 액션
- C. 학습 — 감독 학습으로 쓰임새를, 비용 인식 GRPO로 쓸 때를 익힌다
- D. 증거 — ALFWorld 96.9%·미지 86.6%, 하니스 담금질과 진화의 두 동역학

차례

들어가며	3
01 제1장 — 문제: 하니스를 쓰는 법은 누가 정하나	5
02 제2장 — 설계: 세 상태와 네 액션	8
03 제3장 — 시험대: ALFWorld와 Qwen3-8B	12
04 제4장 — 숫자가 말하는 것	15
05 제5장 — 담금질과 진화: 하니스는 학습되며 바뀐다	20
06 제6장 — 관련 연구와 시리즈의 자리	25
07 맺음말 — 우리 시스템에 남는 것	29

DAY

들어가며



“원문: EvoHarness-RL: Learning Self-Evolving Runtime Harness for Long-Horizon LLM Agents — arXiv:2608.05446 · Xuying Ning, Dongqi Fu, Tianxin Wei, Hanqing Zeng, Yuanchen Bei, Bingxuan Li, Zihao Li, Qifan Wang, Xiang Shen, Yifan Wu, Jiayi Liu, Hong Li, Yinglong Xia, Xiangjun Fan, Hanghang Tong, Jingrui He (University of Illinois Urbana-Champaign · Meta AI)”

“링크: <https://arxiv.org/abs/2608.05446>”

이 책이 읽는 논문

에이전트에 무언가를 엮는 일은 이제 흔하다. 메모리를 붙이고, 도구를 주고, 상태 추적기를 달고, 검증기를 댄다. 그런데 그 모든 장비를 둘러싼 에이전트에게 정작 이런 질문을 던지는 사람은 드물다. **지금 그 하니스를 읽을 때인가, 쓸 때인가, 잠시 무시할 때인가.** 이 논문이 찾은 병목은 바로 그 자리다. 하니스는 갈수록 풍부해지는데, 그 하니스를 쓰는 규칙은 여전히 프롬프트 한 단락과 사람의 휴리스틱으로 남아 있다.

EvoHarness-RL의 답은 단호하다. 하니스 사용법을 프롬프트로 가르치지 말고 **정책으로 학습하라**는 것. 이어지는 구체화도 절제되어 있다. 이질적인 외부 장비를 세 가지 역할 — 환경에 대한 믿음(Belief), 실행의 진척(Progress), 걸쳐온 경험(Experience) — 로 추상화한 BPE 작업 공간을 열고, 에이전트는 track-commit-recall-note 네 가지 메타 액션만으로 그 공간과 대화한다. 학습은 두 단계다. 감독 학습으로 하니스 액션의 쓰임새를 익힌 뒤, 비용을 의식하는 GRPO가 언제 하니스를 만질지 스스로 익히도록 한다.

시험대는 ALFWorld, 모델은 Qwen3-8B다. 결과는 96.9%(본 과제)·86.6%(미지 과제) 성공률이지만, 이 논문의 진짜 수확은 숫자보다 학습이 하니스 사용을 어떻게 바꾸는지에 대한 두 관찰이다. 학습이 오래 걸리는 하니스 습관을 정책 안으로 흡수해 호출을 찾게 쓰다가 선별적으로 쓰게 만드는 **하니스 담금질(harness annealing)**, 그리고 쓰고 버리고 다듬기를 거쳐 경험 창고가 과제에 맞는 콤팩트한 상태로 수렴하는 **하니스 진화(harness evolution)**가 그것이다. 하니스가 고정 장비가 아니라 학습되고 정돈되는 대상임을 보여준다.

시리즈의 세 번째 자리

이 책은 같은 시리즈의 앞 두 권과 한 갈래에서 만난다. 『에이전트는 대화로 일하지 않는다』(SKILL.state 정독)가 하니스 **안쪽**의 실행 상태를, 『하니스의 하니스』(HoH 정독)가 하니스 **바깥쪽**의 유지보수 순환을 다룬다면, 이 논문은 그 사이의 관찰 — 에이전트와 하니스가 서로를 어떻게 조율하는지 — 를 학습 문제로 다룬다. 세 권을 나란히 놓으면 하니스 이야기의 안·바깥·사이가 갖춰진다.

이 책의 읽는 법

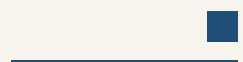
각 장의 머리에 논문의 해당 절을 밝혔다. 논문의 절 구성을 따라가되, 표는 책용으로 다시 조판했고 그림은 원문 그림을 그대로 싣고 한글 해설을 덧붙였다. 표와 그림의 수치는 논문 실측치를 그대로 옮긴 것이고, 제 해석이 들어가는 대목은 장마다 구분해서 밝힌다. 수식은 논문의 표기를 그대로 쓴다.

이 책이 독자에게 바라는 것은 하나이다. 하니스를 “더 좋은 도구”로만 보던 습관을 한 번 접는 것. 그리고 “이 사용법을 누가 정하나”라고 묻는 습관을 얻는 것. 그 물음이 이 책 전체의 결론이기도 하다.

DAY

01

제1장 — 문제: 하니스를 쓰는 법은 누가 정하나



“원문: EvoHarness-RL — arXiv:2608.05446 · §1 Introduction”

긴 실행은 외부에 얹힌다

에이전트가 맡은 일이 길어지자 계산의 무게중심이 모델 밖으로 나왔다. 한 번 답하고 끝나는 과제라면 모델 안에서 다 끝난다. 그러나 몸을 가진 환경과의 상호작용, 웹 탐색, 소프트웨어 공학, 워크플로 자동화 같은 자리에서 에이전트는 오래 버텨야 한다. 환경에 대한 믿음을 유지하고, 끝낸 일과 남은 서브골을 추적하고, 실패한 행동에서 회복하고, 지난 경험의 절차를 다시 꺼내 써야 한다. 이런 짐을 컨텍스트 창 하나로 지탱할 수는 없다. 메모리가 붙고 도구가 달리고, 상태 추적기와 검증기와 실행 로그가 따라온다. 긴 호라이즌(long-horizon) 실행이 갖가지 외부 지원에 의존하게 되는 풍경이다.

여기서 논문은 중심 질문을 꺼낸다. 에이전트는 어떻게 쓸 만한 외부 상태를 스스로 만들고, 그 지원을 자기 결정 과정의 일부로 효율적으로 쓰게 되는가.

외부 하니스라는 층

논문은 이 런타임 층을 외부 하니스(external harness)라 부른다. 프롬프트, 도구, 검색 모듈, 메모리, 상태 추적기, 실행 피드백, 제어 흐름 장치를 두루 묶는 총칭이다. 에이전트 실행을 떠받치는 바깥 장비 전부를 한 단어로 부르는 셈이다.

이 층은 빠르게 풍부해지고 있다. 요즘 에이전트 프레임워크와 제품은 갈수록 정교한 하니스 구성요소를 내놓고, 하니스 엔지니어링 연구는 하니스 상태와 구현, 궤적 기반 적응을 다듬는다. 자기진화 에이전트 연구는 지난 궤적을 재사용 가능한 메모리, 워크플로, 스킬로 증류해 보인다. 장비를 갖추는 일과 장비를 적응시키는 일, 둘 다 저마다 앞서 있다.

병목은 장비가 아니라 사용법

논문이 겨냥하는 병목은 그 반대편에 있다. 외부 구성요소를 아무리 정성껏 설계하고 적응시켜도, 그것을 쓰는 런타임 정책은 프롬프트와 휴리스틱, 도메인 관례로 사람 손에서 정해지는 일이 대부분이다. 초록의 문장이 이 간극을 정확히 찌른다. 에이전트는 유용한 외부 지원에 둘러싸여 있지만, 그 지원을 언제 만들고 접근하고 갱신하고 통합할지는 배운 적이 거의 없다.

이 책의 판단을 붙인다. 도구를 많이 두는 일과 도구를 잘 쓰는 일은 별개의 문제인데, 지금 생태계는 앞의 문제만 열심히 키워 왔다. 하니스가 풍부해질수록 이 간극은 오히려 벌어진다. 고를 수 있는 지원이 늘수록 언제 무엇을 쓸지 정하는 일이 어려워지기 때문이다.

얹힌 두 난제

사용법을 학습 문제로 바꾸려면 두 난제를 넘어야 한다. 논문은 둘이 서로 얹혀 있다고 본다.

하나는 상태 형성(state formation)이다. 상호작용 궤적에는 잡음이 많다. 관찰과 행동, 성공과 실패, 되돌리기가 뒤섞인 원시 기록에서 지금 꺼내 쓸 만한 상태를 뽑아 내야 한다. 필요 없는 디테일은 버리고 살아 있는 사실만 남기는 걸러내기다.

다른 하나는 런타임 통제(runtime control)다. 외부 상태 접근은 공짜가 아니다. 하니스 액션은 환경 액션과 같은 상호작용 예산을 소모한다. 그래서 지금 믿음을 잃을지, 진척을 적을지, 경험을 꺼낼지, 아니면 아무것도 하지 않을지를 매 걸음 결정해야 한다. 매 걸음 하니스를 확인하는 에이전트는 예산을 태우고, 하니스를 외면하는 에이전트는 상태를 잃는다.

둘은 따로 풀 수 없다. 상태를 못 만들면 통제할 거리가 없고, 통제를 못 하면 만든 상태가 예산만 잡아먹는다.

하니스 정책 학습

논문은 이 두 난제를 하나의 문제로 묶어 하니스 정책 학습(harness policy learning)이라 정식화한다. 에이전트가 하니스 정책을 오프라인에서 학습해 두고, 런타임 과제 수행 중에 외부 하니스 상태를 구축하고 갱신하며 쓴다. 하니스 사용이 프롬프트 시점에 박아 넣은 뼈대가 아니라 학습된 런타임 결정이 되는 자리다.

이 문제의 첫 구현이 EvoHarness-RL이다. 이질적인 하니스 구성요소를 세 역할로 추상화해 하나의 작업 공간으로 연다. 환경의 현재 상태를 담은 믿음(Belief), 끝낸 일과 남은 서브골을 적는 진척(Progress), 에피소드를 건너 재사용되는 지식인 경험(Experience). 세 머릿글자를 따 BPE라 부른다. 에이전트는 track-commit-recall-note 네 가지 메타 액션으로 이 공간과 대화한다. 학습은 두 단계다. 감독 하니스 파인튜닝(supervised harness fine-tuning, SFT)이 BPE 액션 프로토콜의 의미와 상태 외재화를 가르치고, 비용을 의식하는 GRPO(cost-aware GRPO)가 과제 성공과 효율, 액션 다양성과 반복 회피에 보상을 주며 언제 읽고 갱신하고 통합할지 탐색하게 한다.

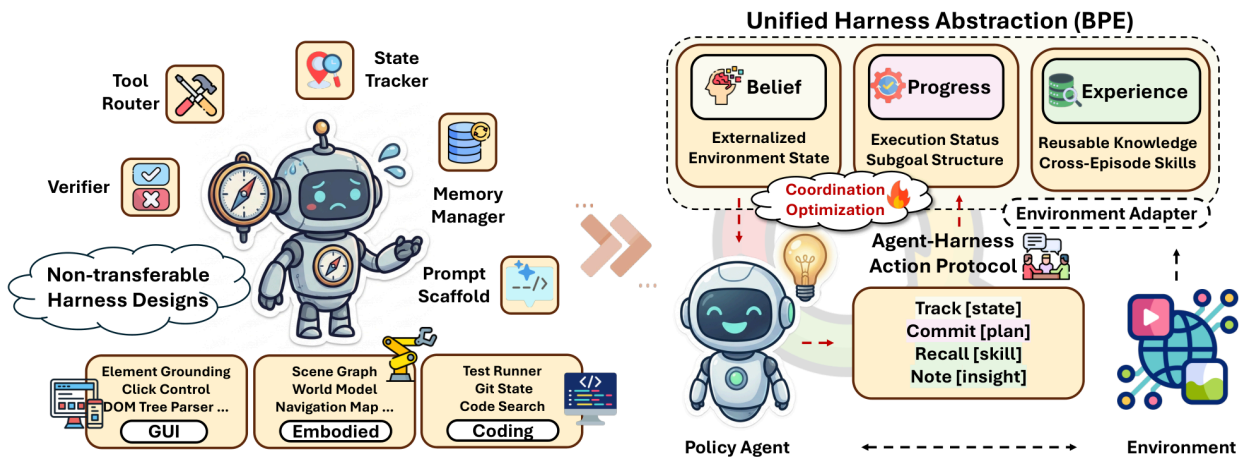


그림 1-1 EvoHarness-RL 개관 — 이질적 외부 하니스를 Belief·Progress·Experience 세 상태로 추상화한다

그림 1-1이 전체 구조를 준다. 긴 호라이즌 에이전트가 의지하는 외부 실행 지원은 원래 제각각이고 사람 손으로 통제된다. EvoHarness-RL은 이 작업 공간을 정책이 마주하는 세 상태로 추상화한다. 환경 상태를 담은 Belief, 실행 현황과 서브골 구조를 담은 Progress, 에피소드를 건너 쓸 지식인 Experience다. 에이전트는 콤팩트한 하니스 액션으로 이 공간을 조율하며, 실행 중 언제 track하고 commit하고 recall하고 note할지를 결정한다. 배우는 대상은 상태 자체가 아니라 그 결정이다.

논문이 걸어 놓은 네 가지

기여는 넷으로 정리된다. 첫째, BPE와 콤팩트한 메타 액션으로 짠 학습 가능한 에이전트-하니스 조율 층을 제안한다. 둘째, 전문가 시연에서 하니스 사용을 부트스트랩한 뒤 GRPO로 비용 인식 조율을 최적화하는 2단계 학습 레시피를 개발한다. 셋째, BPE는 추론과 학습 양쪽에 듣는다는 것을 보인다. 프롬프트 시점 BPE만으로도 상태가 필요한 긴 과제가 좋아지고, Qwen3-8B를 얹은 ALFWorld 실험에서 SFT와 GRPO를 거치면 본 과제 96.9%, 미지 과제 86.6%까지 오른다. 넷째, 학습이 하니스 사용을 어떻게 바꾸는지 두 공진화 동역학으로 분석한다. 반복되는 하니스 사용 패턴이 정책 안으로 흡수되어 잦은 호출에서 선별적 접근으로 옮겨가는 하니스 담금질(harness annealing), 그리고 진척 갱신과 경험 통합을 거쳐 하니스가 콤팩트하고 과제에 맞는 상태 기질로 정돈되는 하니스 진화(harness evolution)가 그것이다.

이 장의 질문으로 돌아가자. 하니스를 쓰는 법은 누가 정하나. 논문의 대답은 사람이 미리 정해 두지 말고 에이전트가 배우게 하라는 것이다. 다음 장에서는 그 학습의 재료가 되는 BPE 작업 공간과 액션 프로토콜을 상세히 연다.

DAY

02

제2장 — 설계: 세 상태와 네 액션



“원문: EvoHarness-RL — arXiv:2608.05446 · §2 Method”

세 실패에서 나온 세 상태

학습 가능한 하니스 인터페이스는 두 요구 사이에서 균형을 잡아야 한다. 긴 지평 실행을 떠날 만큼 외부 상태를 넉넉히 드러내야 하되, 정책 학습이 감당할 만큼 콤팩트해야 한다. 실제 하니스 구현은 상태 추적기, 실행 로그, 과제 계획, 검증기 피드백, 에피소드 기억, 스킬 라이브러리 같은 도메인 특화 구성요소로 흔히 채워진다. 저자들이 주목하는 것은 그 다양함이 아니라, 그 모든 구성요소가 응답하는 실패 모드의 좁은 집합이다. 긴 지평 상호작용에서 에이전트는 셋으로 거듭 무너진다. 환경에서 지금 무엇이 참인지 놓치는 **환경 파악 상실**, 이미 무엇을 마쳤고 다음에 무엇을 시도해야 하는지 잊는 **완료·대기 망각**, 그리고 이전 시도에서 재사용할 수 있었던 절차나 실수를 다시 처음부터 발견하는 **재발견 반복**이 그것이다.

이 세 필요에서 저자들은 정책을 향한 외부 상태의 역할을 세 기능 역할로 묶는다. 환경에 대한 **믿음(Belief)**, 실행의 **진척(Progress)**, 걸쳐온 **경험(Experience)**이다. 각 단계 t 에서 하니스는 하나의 상태를 렌더링한다.

$$\mathcal{H}_t = (B_t, P_t, E_t)$$

세 성분은 각각 환경 추정, 실행 상태, 경험에 대응한다. 믿음 B_t 는 상호작용으로 추론한 과제 관련 사실 — 물건의 상태, 위치, 관계 — 를 저장해 현재 환경에 대한 지속적인 추정을 제공한다. 휘발성 컨텍스트 창 기억만으로 버티지 않아도 되는 이 유다. 진척 P_t 는 부분목표-상태 기록 (g_i, σ_i) 으로 과제 분해와 실행 상황을 남긴다. 무엇을 시도했고 무엇이 열려 있으며 어디서 막혔는지를 밖으로 꺼내어, 암묵적인 추론 자취를 들여다볼 수 있는 과제 상태로 바꾼다. 경험 E_t 는 스킬, 실패 양상, 탐색 사전, 상위 전략 같은 에피소드 간 지식을 유지한다. 실행 중에 관련 있는 지난 경험을 꺼내 주고 새 통찰을 저장해 뒤늦은 통합에 넘긴다.

네 액션 — 읽고 쓰는 최소 문법

상태가 갖춰져도 정책은 그 작업 공간을 읽고 쓸 간결한 수단이 필요하다. 여기서 갈림길이 둘 나온다. 완전히 도메인 특화된 하니스 API는 많은 연산을 노출할 수 있지만, 학습된 행동이 다른 곳으로 옮겨 가거나 분석되기 어려워진다. 반대로 범용 메모리 액션 하나로 모든 것을 처리하면 작업 공간의 기능적 구조가 액션 뒤에 숨어 버린다. 저자들은 두 극단 사이에서, 에이전트와 BPE 사이의 주요 정보 흐름을 덮는 작은 메타 액션 집합을 정의한다.

$$\mathcal{A}_{\text{bpe}} = \{\text{track}, \text{commit}, \text{recall}, \text{note}\}$$

대응은 단정적이다. track은 B_t 에서 과제 관련 믿음을 읽는다. commit은 부분목표 또는 실행 갱신을 P_t 에 쓴다. recall은 E_t 에서 재사용 가능한 지식을 꺼낸다. note는 새 통찰을 기록해 뒤늦은 경험 통합에 넘긴다. 상호작용 중 정책은 환경 액션과 하니스 액션을 한 묶음에서 고른다.

$$\mathcal{A} = \mathcal{A}_{\text{env}} \cup \mathcal{A}_{\text{bpe}}$$

단계 t 에서 정책은 환경 관찰 o_t 와 렌더링된 하니스 상태 $\mathcal{H}_t$, 과제 맥락 c_t 를 받아 다음 액션을 뽑는다.

$$a_t \sim \pi_{\theta}(\cdot \mid o_t, \mathcal{H}_t, c_t)$$

$a_t \in \mathcal{A}_{\text{env}}$ 이면 환경이 앞으로 나가고 새 관찰이 돌아온다. $a_t \in \mathcal{A}_{\text{bpe}}$ 이면 외부 작업 공간을 조회하거나 갱신하고 새 하니스 뷰가 돌아온다. 눈여겨 읽을 대목은 그 다음 문장이다. 두 액션 유형이 **같은 상호작용 예산을 소모한다**는 것. 하니스를 한 번 읽는 일도 환경을 한 번 움직이는 일과 같은 예산을 쓰니, 하니스 접근이 언제 그 비용을 치르고도 남는가는 정책이 학습해야 할 문제가 된다. 그 조율의 최적화는 이 장 마지막 절의 몫이다.

어댑터 — 같은 인터페이스, 다른 내부

BPE는 고정된 내부 스키마가 아니라 기능 인터페이스다. 과제마다 필요한 하니스 구현은 달라도, 정책을 향해 드러내는 역할은 믿음·진척·경험으로 같을 수 있다. 이 다리를 놓는 것이 환경 어댑터다. 어댑터는 관찰과 액션 결과, 도구 출력, 검증기 피드백을 처리하고 내부 저장소를 유지하다가, 정책에게는 선택된 뷰를 (B_t, P_t, E_t)로 내놓는다. 네 하니스 액션도 대상 환경에 맞춰 착지시킨다. 내부 구현은 도메인 특화로 남고, 학습되는 에이전트-하니스 조율층은 공유된다.

시험대 ALFWorld에서의 착지는 이렇다. 믿음은 환경이 한 걸음 나갈 때마다 뒤에서 갱신되는 내부 세계 상태 저장소다. 물건 상태와 위치, 공간 관계 같은 과제 관련 사실을 에이전트의 액션과 관찰로 유지한다. 이 상태는 기본으로 전부 열리지 않는다. 정책이 `track[object]`로 특정 물건을 들여다보거나 `track[world]`로 콤팩트한 전역 요약을 받는, 선택적 하니스 액션의 형태로만 접근된다.

진척은 커밋된 실행 기록이다. ALFWorld에서는 부분목표-상태 항목의 길이 제한 목록으로 구현되며, 대체로 순차적인 가사 과제에 족하다. 예컨대 청소 후 배치 과제는 대상 물건 찾기, 집기, 청소하기, 목표 용기에 놓기로 나뉜다. 정책은 `commit[subgoal]`로 지금 걸음을 밖에 남기고, 시도됨·대기·차단이 이후 판단에 보이게 된다.

경험은 에피소드를 건너는 스킬 저장소다. 일반 스킬, 과제별 스킬, 흔한 실수, 물건 위치 탐색 사전으로 조직되고 두 시간 규모로 진화한다. 에피소드 안에서는 `recall[query]`가 관련 지식을 꺼내며 회수된 항목의 사용 횟수를 갱신하고, `note[insight]`가 새 교훈을 임시 노트 버퍼에 쓰며, 성공한 물건 탐색은 위치 사전을 즉시 갱신한다. 병렬 롤아웃 수집 동안 주 스킬 저장소는 배치 안에서 고정되고, 노트 버퍼와 완료 궤적 요약이 뒤에서 쌓인다. 에포크 경계에 이르면 통합 모델이 이 증거를 `add·update·remove` 연산으로 스킬 저장소에 합친다.

두 단계 학습 — 모방에서 비용 의식으로

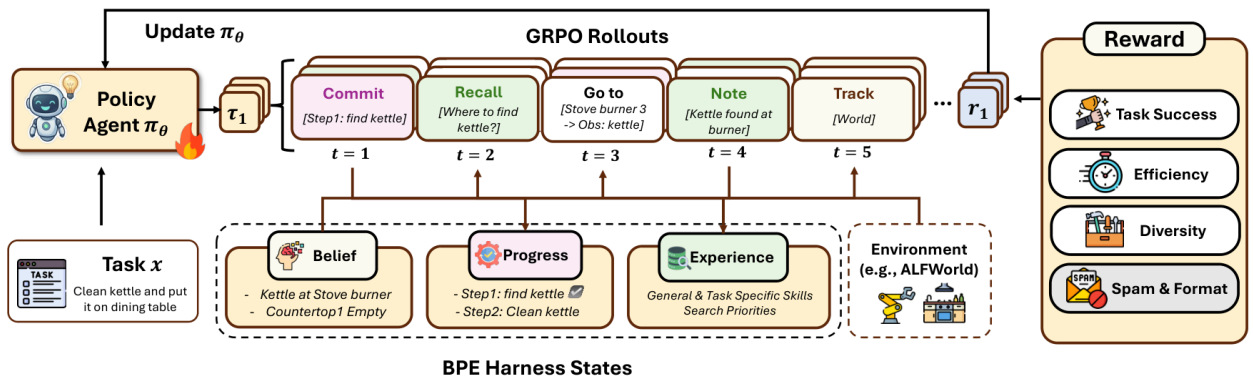


그림 2-1 학습 파이프라인 — SFT 체크포인트에서 GRPO 롤아웃, 보상은 성공·효율·다양성·벌점의 합

학습은 두 단계다. 첫 단계는 같은 BPE 인터페이스로 수집한 성공 교사 궤적 위의 감독 미세조정(SFT)이다. 교사는 매 걸음 과제 목표, 현재 관찰, 허용된 환경 액션, 최근 이력, 활성 하니스 뷰를 보고 `<think>...</think><action>...</action>` 형식으로 다음 액션 하나를 낸다. 그 액션은 ALFWorld 명령일 수도, BPE 하니스 액션일 수도 있다. 저자들은 Qwen3-8B를 이 다음-액션 시연으로 미세조정하며, 과제 해결 행동과 함께 `track·commit·recall·note`를 언제 쓰는지의 기초 의미를 가르친다. 교사 롤아웃 동안 쌓인 경험은 이어지는 GRPO가 쓸 스킬 저장소를 초기화한다.

둘째 단계는 SFT 체크포인트에서 시작하는 GRPO다. 궤적 수준 보상 $R(\tau)$ 은 지배적인 회소 성공 신호 위에, 적응적 하니스 사용을 기르는 촘촘한 보조 항을 엮는다.

$$R(\tau) = \underbrace{R_{\text{succ}}(\tau)}_{\text{task success}} + \underbrace{\lambda_{\text{eff}} R_{\text{eff}}(\tau)}_{\text{efficiency bonus}} + \underbrace{\lambda_{\text{div}}(u) R_{\text{div}}(\tau)}_{\text{action diversity}} - \underbrace{\lambda_{\text{spam}} R_{\text{spam}}(\tau)}_{\text{spam penalty}} - \underbrace{\lambda_{\text{inv}} R_{\text{inv}}(\tau)}_{\text{format penalty}}$$

과제 완수가 엄격한 문지기 노릇을 한다. $R_{\text{succ}}(\tau) = 10 \cdot \mathbf{1}[\text{solved}]$ 이고, 효율 보너스 $R_{\text{eff}}(\tau) = \max(0, 1 - |\tau|/T_{\text{max}})$ 는 오직 성공했을 때만 지급된다. 성공해야 받고 궤적이 길수록 얇아지니, 잉여로운 하니스 질의는 저절로 억눌린다.

붕괴 방지가 다음 고민이다. 정책이 $\mathcal{A}_{\text{bpe}}$ 를 통째로 무시하거나, 반대로 같은 액션을 끝없이 되풀이하는 두 붕괴를 막으려 저자들은 시간 의존 어휘 다양성 보너스를 넣는다.

$$R_{\text{div}}(\tau) = \frac{|\{\text{verb}(a_t) : a_t \in \tau\}|}{|\tau|}, \quad \lambda_{\text{div}}(u) = \frac{\lambda_{\text{div}}^{\max}}{2} \left(1 + \cos \frac{\pi u}{U}\right)$$

궤적에 등장한 서로 다른 동사의 수를 궤적 길이로 나눈 것이 R_{div} 다. 가중치 식에서 u 는 현재 RL 에포크, U 는 담금질 시한이다. 코사인 곡선을 따라 초기에는 무겁게 붙었다가 훈련이 깊어지면 우아하게 줄어든다. 이 커리큘럼은 훈련 초반에 하니스 액션을 넓게 탐색하게 한 뒤 점차 압력을 거두어, 전문화와 효율적 과제 해결로 몰아간다. 마지막으로 R_{spam} 과 R_{inv} 는 무의미한 반복과 형식이 틀어진 액션에 고정 벌점을 매긴다.

성공이 아니면 보너스가 없고, 성공해도 길면 보너스가 줄고, 초반의 다양성 권장은 시간이 지나며 걷히고, 군더더기에는 벌금이 붙는다. 보상 설계 한 장에 “하니스는 공짜가 아니다”라는 이 논문의 전제가 그대로 새겨져 있다.

DAY

03

제3장 — 시험대: ALFWorld와 Qwen3-8B

“원문: EvoHarness-RL — arXiv:2608.05446 · §3.1 실험 설정, Appendix C 구현 세부(표 4)”

실험 논문의 설득력은 절반쯤 시험대에서 결정된다. 너무 어려우면 모두가 바닥을 걷고 너무 쉬우면 모두가 천장에 닿는다. 어느 쪽이든 방법의 차이는 지워진다. 차이를 보이려면 신호가 움직일 여백이 시험대에 있어야 한다. 이 장은 무대 ALFWorld, 상대 세 갈래 베이스라인, 주역 Qwen3-8B를 짚고 Appendix C에서 실험의 손맛을 읽는다.

시험 환경 ALFWorld — 여섯 과제군의 가정 집

ALFWorld(Shridhar et al., 2021)는 embodied AI 벤치마크 ALFRED와 정합하는 텍스트 기반 게임이다. 에이전트는 텍스트 명령으로 방을 오가며 물건을 조작해 다단계 가사 과제를 완수한다. 논문은 상태 추적 요구가 제각기 다른 여섯 과제군에 주목한다. 물건을 집어 옮기기(Pick), 조명 아래 살펴보기(Look), 씻어서 놓기(Clean), 데워서 놓기(Heat), 식혀서 놓기(Cool), 두 물건 놓기(Pick2)가 그 여섯이다. 성공률은 Ouyang et al. (2026) 관례의 표준 검증 세트에서 보고한다. 에이전트가 기억해야 할 것은 과제군마다 다르다. Pick은 찾아서 집어 놓는 순서만 물으면 되지만, Heat는 데우고 꺼내 옮기기까지의 중간 상태를 기억해야 하고, Pick2는 두 대상의 진행을 따로 헤아려야 한다. 신념 추적기와 진행 기록이 감당할 무게가 과제군마다 달라진다는 뜻이다. 성공률을 과제군별로 보고해 하니스의 이득이 어느 상태 추적에서 나오는지 읽힌다.

세 갈래 베이스라인, 그리고 본 변형 세 가지

비교군은 세 범주로 갈린다. 프론티어 모델 Claude Opus 4.5, GPT-4.1, GPT-5는 표준 프롬프팅과 프롬프트 타임 하니스 각각으로 평가해 강한 기반 정책이 명시적 BPE 구조에서 얻는 이득을 잔다. 파라미터를 열린 채 외부 메모리나 경험 풀에 기대는 표준 방법으로는 ReAct, ExpeL, ReasoningBank, MemP, Dynamic Cheatsheet, ACE, SkillOS-base가 서고, 메모리 기제나 구조 최적화를 학습 안에 녹이는 트레이너를 방법으로 표준 GRPO, SkillOS, SkillRL이 있다.

맞은편의 본 변형은 세 가지다. 하니스를 추론 시점에만 엮는 EvoHarness-Base, 교사 궤적에서 하니스 사용법을 지도 학습하는 EvoHarness-SFT, 비용 인식 GRPO로 정책과 하니스 사용을 함께 조율하는 EvoHarness-RL. 같은 하니스를 점점 깊이 결합하는 설계라 하니스 구조의 이득과 학습의 이득을 나눠 읽는다.

Appendix C — 구현을 열어 보면

학습되는 모델은 Qwen3-8B 하나뿐으로 SFT와 GRPO에서 갱신되며 롤아웃 루프 안에서 이 모델만 실행된다. 정책은 매 단계 `<think>...</think><action>...</action>` 형식으로 응답하고 파싱된 액션은 ALFWorld 또는 BPE 하니스로 보내진다. SFT 궤적 수집과 경험 저장소 통합의 교사는 Claude Opus다. 다만 통합은 에포크 경계에 루프 밖에서 돌아가므로 롤아웃은 환경과 현재 하니스를 상대한 Qwen3-8B의 단독 상호작용으로 남는다.

환경 래퍼는 `<action>` 태그 안의 내용을 꺼내 먼저 ALFWorld 허용 명령과 대조하고, 어긋나면 `track-commit-recall-note` 하니스 패턴과 대조한다. 둘 다 아닌 출력은 실패 피드백과 함께 무효 액션 처벌로 세인다. 베이스 트레이너의 내장 처벌은 끄고 보상 설계는 전부 자체 보상 함수에 맡긴다. 그라운딩은 될 수 있는 대로 결정론적이다. 신념 추적기는 LLM 호출 없이 액션-관측 쌍을 읽는 규칙 기반 파서고, 진행 추적기는 정책이 `commit`을 내놓을 때만 갱신되는 유계의 목록이며, 경험 저장소는 키워드 겹침으로 회상하고 `note` 출력을 버퍼에 쌓아 통합에 넘기며 집기에 성공하면 위치 우선지도를 갱신한다. 표 4의 수치로는 에피소드 길이 상한 $T_{\max} = 70$, 신념 예지 48, 부분목표 상한 8, 범주별 회상 top-3이며, 경험은 General-task-specific-mistakes-search priors 범주로 범주당 80까지, 퇴출은 LFUD다.

SFT 데이터는 교사가 같은 액션 인터페이스로 훈련 게임 500개를 풀어 성공 에피소드만 남긴 것이다. 궤적 87개, 대화쌍 1,153쌍, 평균 26.5턴이고 과제군별로는 pick-two 28, pick-place 17, clean 15, heat 11, light 8, cool 8로 여섯 군을 다 덮는다. 입력은 과제 목표·현재 관측·허용 명령·최근 이력·활성 하니스 뷰이고 정답은 교사의 다음 `<think>`와 `<action>`이다. 교사의 하니스 호출은 405회로 전체 턴의 약 18퍼센트고 `commit` 202, `recall` 114, `note` 55, `track` 34다. 이 체크포인트가 GRPO의 출발점이 되고 수집 중 쌓인 경험이 초기 스킬 뱅크가 된다.

GRPO는 프롬프트마다 여러 궤적을 뽑아 보상을 그룹 안에서 정규화하고 SFT 참조에 대한 KL 벌점을 걸고 정책을 움직인다. 롤아웃 동안 궤적 요약과 note 버퍼는 쌓이기만 하다가 에포크가 끝나면 통합 모델이 저장소를 갱신한다. 배치 안의 스킬 뱅크를 흔들지 않으면서 하니스의 에피소드 간 진화를 여는 장치다. 규모는 그룹 크기 8, 단계당 프롬프트 16·궤적 128, 총 에포크와 담금 지평 150, AdamW 학습률 1×10^{-6} , KL 계수 0.01, vLLM에 H200 8장이다.

이 장의 읽을거리 — 왜 Qwen3-8B인가

시험대의 마지막 조각은 정책 모델의 크기다. 이 실험이 잡으려는 신호는 학습이 하니스 사용을 바꾸는 모습인데 프론티어에는 그 신호가 없다. 표 1에서 Claude Opus 4.5는 표준 ReAct만으로 평균 96.4에 있다. 상승분이 드러나려면 실패할 여지가 충분해야 하는데 Qwen3-8B는 그 여백 안에 서 있는 크기다.

둘째는 롤아웃 구조다. 루프 안에 Qwen3-8B만 있으므로 하니스가 좋아져도 성적이 오르는 유일한 길은 정책 파라미터가 바뀌는 것이다. 에포크 150에 궤적 128씩 감당할 현실적 크기이기도 하다. 하니스가 학습되고 정돈되는 대상임을 보이려면 학습될 수 있는 모델이 필요했던 셈이다.

축	선택	시험이 노리는 것
환경	ALFWorld 여섯 과제군	상태 추적 부담의 변주
비교	프론티어·프존·트레이너블	하니스 이득과 학습 이득의 분리
정책	Qwen3-8B	학습 신호가 보일 여백

시험대는 그래서 두 겹이다. 가정 집이 과제의 변주를 주고 8B 모델이 학습의 여백을 준다. 이 무대의 결과가 다음 장 본론이다.

DAY

04

제4장 — 숫자가 말하는 것



“원문: EvoHarness-RL — arXiv:2608.05446 · §3.2–3.4 (주 결과·절제·일반화), Tables 1–3”

설계를 봤으니 이제 성적표다. 이 장은 논문의 세 표 — 주 결과(Table 1), 성분 절제(Table 2), 일반화(Table 3) — 를 수치 하나 바꾸지 않고 옮긴 뒤, 표 아래에서 한 줄씩 읽는다. Pick, Look, Clean, Heat, Cool, Pick2는 ALFWorld의 여섯 과제군 이름이다. Pick은 물건을 찾아 정해진 자리에 두는 과제이고 Pick2는 그것을 두 개로 늘린 것이며, Clean·Heat·Cool은 세척·가열·냉각이라는 상태 변화를 다루고, Look은 탐색과 관찰 과제다.

주 결과 — 140과제의 성공률

먼저 전체 표를 옮긴다. seen 분할 140과제의 성공률이고, Δ는 같은 백본의 ReAct 기준선 대비 평균 성공률의 절대 이득이다.

구분	방법	백본 · 변형	Pick	Look	Clean	Heat	Cool	Pick2	평균	Δ
프론티어	ReAct	Opus 4.5*	100.0	92.3	96.3	100.0	88.0	100.0	96.4	–
	1. EvoHarness-Base	Opus 4.5*	100.0	100.0	100.0	93.8	96.0	100.0	98.5	+2.1
	ReAct	GPT-4.1*	82.9	61.5	44.4	43.8	4.0	41.7	47.9	–
	1. EvoHarness-Base	GPT-4.1*	82.9	61.5	63.0	75.0	72.0	62.5	70.0	+22.1
	ReAct	GPT-5*	74.3	53.8	48.1	62.5	60.0	58.3	60.7	–
	1. EvoHarness-Base	GPT-5*	97.1	76.9	85.2	93.8	88.0	62.5	85.0	+25.7
소형 모델	ReAct	Qwen3-8B*	78.1	46.2	33.3	37.5	29.3	47.2	47.9	–
	ExpeL	Qwen3-8B*	91.4	76.9	14.8	43.8	28.0	45.8	49.3	+1.4
	ReasoningBank†	Qwen3-8B*	83.8	48.7	49.4	39.6	41.3	54.2	55.7	+7.8
	MemP†	Qwen3-8B*	80.0	43.6	24.7	33.3	38.7	48.6	49.7	+1.8
	Dynamic Cheatsheet	Qwen3-8B*	88.6	53.8	29.6	37.5	20.0	66.7	52.1	+4.2
	ACE	Qwen3-8B*	85.7	38.5	29.6	37.5	36.0	58.3	51.4	+3.5
	SkillOS-base†	Qwen3-8B*	79.0	41.0	45.7	37.5	38.7	55.6	53.1	+5.2
	GRPO	Qwen3-8B▲	87.5	71.4	72.7	70.0	48.1	43.5	65.6	+17.7
	SkillOS†	Qwen3-8B▲	95.2	71.8	74.1	72.9	77.3	77.8	80.2	+32.3
	SkillRL‡	Qwen2.5-7B▲	97.9	71.4	90.0	90.0	95.5	87.5	89.9	+42.0
본 기법	EvoHarness-Base	추론 시점*	71.4	53.8	63.0	50.0	48.0	41.7	56.4	+8.5
	EvoHarness-SFT	학습 호출▲	80.0	53.8	88.9	75.0	40.0	62.5	68.6	+20.7
	EvoHarness-RL	SFT→GRPO▲	100.0	92.9	95.5	100.0	92.6	100.0	96.9	+49.0

*는 동결된 추론 시점 방법, ▲는 학습이 필요한 방법이다. †와 ‡는 각각 SkillOS와 SkillRL 논문이 보고한 값을 그대로 인용한 수치다. 백본 · 변형 열은 지면 폭 확보를 위해 표기를 줄였다 — Opus 4.5는 Claude Opus 4.5, SFT→GRPO는 SFT 초기화 후 GRPO 최적화다.

논문의 주장은 맨 아래 행에 압축돼 있다. Qwen3-8B에 EvoHarness-RL을 얹으면 평균 96.9%로 표 전체의 최고 성능이 나오고, ReAct 기준선 47.9% 대비 +49.0점이다. 이 최적화 덕분에 8B 모델이 Claude Opus 4.5 같은 최상위 프론티어와 실질적으로 대등해진다는 것이 논문의 요지다. 동결 메모리 기법은 물론 학습형 기법인 SkillOS(80.2%, +32.3)와 SkillRL(89.9%, +42.0)까지 전부 앞선다. 본 변형의 세 행이 만드는 계단 — 프롬프트 시점 스캐폴딩 56.4%, SFT

68.6%, GRPO 96.9% — 이 2단계 훈련 파이프라인을 그대로 검증한다는 것, 최적화가 하니스를 정적 도구에서 효과적인 의사결정 인터페이스로 바꿨다는 것이 논문이 내리는 결론이다.

셀 단위로 다시 읽으면 이 숫자의 성격이 더 분명해진다. EvoHarness-RL은 Pick-Heat-Pick2에서 100.0%를 찍고 가장 낮은 Look조차 92.9%다. 여섯 과제군 전체에서 고르게 높다는 뜻이지 한두 과제군의 특기가 평균을 끌어올린 것이 아니다. 오픈소스 블록과의 격차도 평균만큼이나 넓다. 가장 강한 동결 기법인 ReasoningBank(55.7%)와의 차이는 41.2점이고, 같은 백본을 쓰는 SkillOS†(80.2%)와도 16.7점 벌어진다. 표 전체에서 평균 90%를 넘긴 행은 프론티어의 Claude Opus 4.5+Base(98.5%)와 EvoHarness-RL 둘뿐이고 학습형 경쟁 기법 중 최고인 SkillRL†는 89.9%로 그 문턱에 못 미친다.

가중치를 건드리지 않고 오르는 프론티어

표 위쪽 세 쌍은 학습이 전혀 없는 실험이다. 같은 모델에 프롬프트만으로 BPE 하니스를 얹었을 때 무슨 일이 나는지를 본다. GPT-4.1은 47.9%에서 70.0%(+22.1)로 오르고 GPT-5는 60.7%에서 85.0%(+25.7)로 오른다. 성능 상한 근처에 있던 Claude Opus 4.5조차 96.4%에서 98.5%(+2.1)로 밀려 올라간다. 논문은 이를 근거로 belief-progress-experience를 모델 밖에 두는 일이 기반 모델의 크기와 무관하게 장기 과제 실행의 핵심이라고 읽는다.

셀로 들어가면 이득의 출처가 보인다. 가장 극적인 셀은 GPT-4.1의 Cool이다. 4.0%에서 72.0%로 68점이 오른다. 냉각처럼 물건의 상태 변화를 몇 단계에 걸쳐 추적해야 하는 과제에서는 프론티어라도 상태를 모델 안 기억에만 맡기면 무너지며, 외부 상태판이 바로 그 구멍을 메운다는 뜻이다. 같은 모델의 Clean도 44.4%에서 63.0%로 오른다. 반대 방향의 흔적도 있다. GPT-5의 Pick2는 58.3%에서 62.5%로 거의 제자리고, Claude Opus 4.5의 Heat는 100.0%에서 93.8%로 내려간다 — 이 표에서 유일하게 하락한 셀이다. 두 물건을 조합하는 순차 계획의 병목은 프론티어에서 프롬프트 하니스가 거의 풀지 못했고, 이미 만점인 영역에서는 하니스가 오히려 간섭이 될 수 있다는 것이 함께 읽힌다.

세 계단 — 56.4, 68.6, 96.9

나머지 이야기의 중심은 맨 아래 세 행이다. 같은 Qwen3-8B 위에서 하니스를 프롬프트로만 얹으면 56.4%(+8.5), 호출 방법을 감독 학습하면 68.6%(+20.7), SFT 초기화에 GRPO 최적화까지 더하면 96.9%(+49.0)가 된다. 세 계단의 성격이 각각 다르다. 첫 계단은 구조를 주는 일이고 둘째 계단은 쓰는 법을 흉내 내는 일이며 셋째 계단은 결과를 보고 부리는 법을 익히는 일이다. 논문의 표현을 빌리면 하니스가 정적 도구에서 의사결정 인터페이스로 바뀌는 지점이 셋째 계단이다.

오픈소스 블록과 견주면 이 계단의 위치가 더 명확해진다. 메모리를 쌓는 동결 기법은 +1.4(ExpeL)부터 +7.8(ReasoningBank)까지 한 자릿수에 머문다. 학습이 들어가는 순간 GRPO 단독 +17.7, SkillOS +32.3, SkillRL +42.0까지 올라간다. 그리고 하니스 구조와 2단계 학습을 결합한 EvoHarness-RL이 +49.0으로 이 계열을 닫는다. 무엇을 밖에 둘 것인가와 그것을 언제 쓸지 학습할 것인가가 겹쳐야 이득이 두 자릿수 후반으로 넘어간다는 구도다.

소형 모델과 프론티어의 자리바꿈도 이 표에서 확인된다. 96.9%는 동결 프론티어 전부를 넘는다. Claude Opus 4.5 ReAct(96.4%), GPT-5+Base(85.0%), GPT-4.1+Base(70.0%)가 그 아래에 있다. 하니스까지 얹은 가장 강한 프론티어 조합인 Claude Opus 4.5+Base(98.5%)에만 1.6점 미달한다. 8B 모델이 학습만으로 프론티어의 동결 성능을 넘어선다는 사실은 장기 과제 성능의 상당 부분이 모델 용량이 아니라 상태 관리에서 나온다는 방증이다.

한 조각씩 빼면 — 절제 실험

다음 표는 하니스의 세 성분을 동결 Qwen3-8B의 추론 시점 하니스에서 하나씩 뺀 실험이다. 위쪽 두 행은 비교를 위한 학습 정책의 참고치다.

구분	변형	인터페이스 · 정책	Pick	Look	Clean	Heat	Cool	Pick2	평균
참조 · 학습 정책	EvoHarness-SFT	전체 BPE · SFT	80.0	53.8	88.9	75.0	40.0	62.5	68.6

구분	변형	인터페이스 · 정책	Pick	Look	Clean	Heat	Cool	Pick2	평균
	EvoHarness-RL	전체 BPE · SFT→GRPO	100.0	92.9	95.5	100.0	92.6	100.0	96.9
동결 8B 추론 시점 절제	EvoHarness-Base	Belief + Progress + Experience	71.4	53.8	63.0	50.0	48.0	41.7	56.4
	w/o Belief	객체 · 상태 추적 제거	74.3	53.8	40.7	75.0	20.0	37.5	50.0
	w/o Progress	부목표 · 진행 추적 제거	68.6	53.8	44.4	68.8	32.0	37.5	50.7
	w/o Experience	회상 · 메모 · 스킬뱅크 제거	65.7	53.8	40.7	62.5	28.0	41.7	48.6

논문의 읽기는 성분별로 정리된다. Belief를 빼면 객체 추적이 사라져 위치 파악과 상태 검증이 필요한 Clean-Cool이 무너진다. Progress를 빼면 부목표에 대한 커밋이 사라져 의존 관계의 부목표가 이어지는 장기 과제가 다친다. Experience를 빼면 스킬 회상과 실수 회피가 사라져 평균 48.6%로 절제 실험 전체의 최저가 된다. 어느 하나가 빠져도 실행이 크게 나빠지므로 세 성분은 따로 노는 메모리 트릭이 아니라 하나로 묶인 상태 인터페이스로 작동한다는 것이 논문의 결론이다.

이 표의 셀을 원문 서술과 하나씩 맞춰보면 결이 조금 다르다. Belief 절제에서 Cool 48.0→20.0, Clean 63.0→40.7의 급락은 논 의 그대로다. 그런데 같은 변형의 Heat는 50.0→75.0, Pick은 71.4→74.3으로 오히려 오른다. Experience 절제에서 논문이 상태 변화 과제의 타격 사례로 든 Heat는 50.0→62.5로 상승해 있고 실제 하락은 Clean(→40.7)과 Cool(→28.0)에 몰려 있다. Progress 절제가 Pick2를 특히 다친다는 서술도 수치와는 어긋난다. Pick2 하락은 41.7→37.5로 4.2점인데 같은 변형의 평균 하락 5.7점보다 작고 이 변형의 최대 타격은 Cool(→32.0)이다. Look은 네 절제 변형 전부에서 53.8%로 열어 있으니 관찰 중심 과제는 이 하니스의 어느 성분에도 의존하지 않았다. 요약하면 세 성분 전부 필요하다는 평균 수준의 결론은 수치로 확실히 뒷받침되지만(56.4→48.6~50.7), 어느 성분이 어느 과제군을 지탱하는지에 대한 원문의 서술은 셀 단위로는 상당 부분 맞지 않는다. 과제군별 표본이 크지 않아 셀 변동의 상당수가 노이즈 범위일 수 있다는 점도 함께 감안해야 한다.

처음 보는 과제에서 — 일반화

마지막 표는 학습 중에 본 적 없는 unseen 분할에서의 성공률이다. 같은 Qwen3-8B로 ReAct, 프롬프트 시점 하니스, SFT, GRPO 최적화 네 단계를 나란히 놓는다.

방법	백본 · 변형	Pick	Look	Clean	Heat	Cool	Pick2	평균
ReAct	Qwen3-8B*	66.7	50.0	48.4	47.8	52.4	29.4	50.0
EvoHarness-Base	프롬프트 시점*	83.3	38.9	87.1	91.3	90.5	58.8	77.6
EvoHarness-SFT	학습 호출▲	70.8	66.7	80.6	69.6	47.6	76.5	69.4
EvoHarness-RL	SFT→GRPO▲	70.8	94.4	87.1	87.0	95.2	88.2	86.6

*는 동결 방법, ▲는 학습 방법이다.

ReAct가 50.0%인 자리에서 프롬프트 시점 하니스가 77.6%까지 끌어올린다. 상태를 밖에 두기만 해서 얻는 이득이 27.6점이다. 그다음이 흥미롭다. 학습이 들어간 SFT는 69.4%로 프롬프트 하니스보다 아래로 내려간다. 논문은 감독 모방이 seen 트랙토리의 교사 패턴을 배울 뿐 낯선 환경에서 하니스에 접근할 가치가 있는 시점까지는 최적화하지 못하기 때문이라고 진단한다. 반면 GRPO까지 더한 정책은 86.6%로 unseen에서도 최고를 유지한다. 비용을 의식하는 GRPO가 하니스 접근을 재보정하여 훈련 환경을 외우는 대신 두루 통하는 전략을 배웠다는 것이 논문의 해석이다.

seen(앞의 표)과 unseen을 나란히 놓으면 변형별 방향이 갈라진다. Base는 56.4%에서 77.6%로 21.2점 오르고 SFT는 68.6%에서 69.4%로 제자리이며 RL만 96.9%에서 86.6%로 10.3점 내려간다. 기준선 ReAct는 47.9%에서 50.0%가 되니 두 분할의 난이도 자체는 엇비슷하고 차이는 방법에서 온 것이다. Base가 처음 보는 과제에서 오히려 크게 오른다는

것은 프롬프트 하니스의 이득이 과제 익숙함과 무관하게 작동한다는 뜻이다. SFT가 seen에서만 Base를 앞지른 것은 모방의 대상이 훈련 분할에 붙어 있었다는 원문의 진단과 방향이 맞는다. RL의 하락은 셀로 보면 Pick에 집중된다. seen에서 100.0%였던 Pick이 unseen에서 70.8%인 반면 Look은 92.9%에서 94.4%로, Cool은 92.6%에서 95.2%로 오히려 오른다. seen에서의 완벽이 과제 특이적 암기에 가까웠을 가능성을 남기는 흔적이지만 하락한 뒤에도 86.6%는 2위 Base(77.6%)와 9.0점 차이이다. unseen 표에도 역행 셀은 있다. Base의 Look 38.9%는 같은 열의 ReAct(50.0%)보다 낮은 유일한 셀로 프롬프트 하니스가 관찰 중심 과제에서는 간섭이 된다는 신호인데 이 신호는 학습을 거친 두 변형에서 사라진다.

■ 숫자가 남기는 것

정리하면 이렇다. 세 계단 56.4→68.6→96.9는 학습이 하니스를 도구에서 의사결정 인터페이스로 바꾸는 과정이었다. 프론티어 세 쌍(+2.1, +22.1, +25.7)은 가중치를 건드리지 않고도 상태 외부화의 이득이 모델 규모를 가리지 않음을 보여줬다. 절제 표는 세 성분 중 하나만 빼도 평균이 48.6~50.7%로 무너진다고 말했고 일반화 표는 처음 보는 과제에서도 86.6%로 8B 학습 모델이 프론티어의 동결 성능을 넘는다고 말했다. 세 표가 가리키는 결론은 하나로 모은다. 장기 과제의 병목은 모델의 크기가 아니라 상태를 모델 안에만 둘 것인가 밖에서 다룰 것인가였다.

DAY

05

제5장 — 담금질과 진화: 하니스는 학습되며 바뀐다



“원문: EvoHarness-RL — arXiv:2608.05446 · §4 Analysis, Appendix A”

벤치마크 표가 성능의 결말을 보여준다면, 분석 장은 성능이 만들어지는 과정을 보여준다. 논문은 EvoHarness-RL의 학습 과정에서 서로 다른 두 겹의 동역학을 분리해 들여다본다. 하나는 정책 쪽에서 일어나는 **하니스 담금질(harness annealing)**로, 학습이 되풀이되는 하니스 사용을 정책 안으로 걸어 들이는 현상이다. 다른 하나는 하니스 쪽에서 일어나는 **하니스 진화(harness evolution)**로, 경험 저장소가 에피소드를 거둬들여 스스로 모양을 바꾸는 현상이다. 이 장은 본문 §4의 두 분석을 먼저 읽고, 부록 A의 액션별 분해와 보상 비교를 더해 마무리한다.

하니스 담금질 — 잦은 호출에서 한 번의 선별로

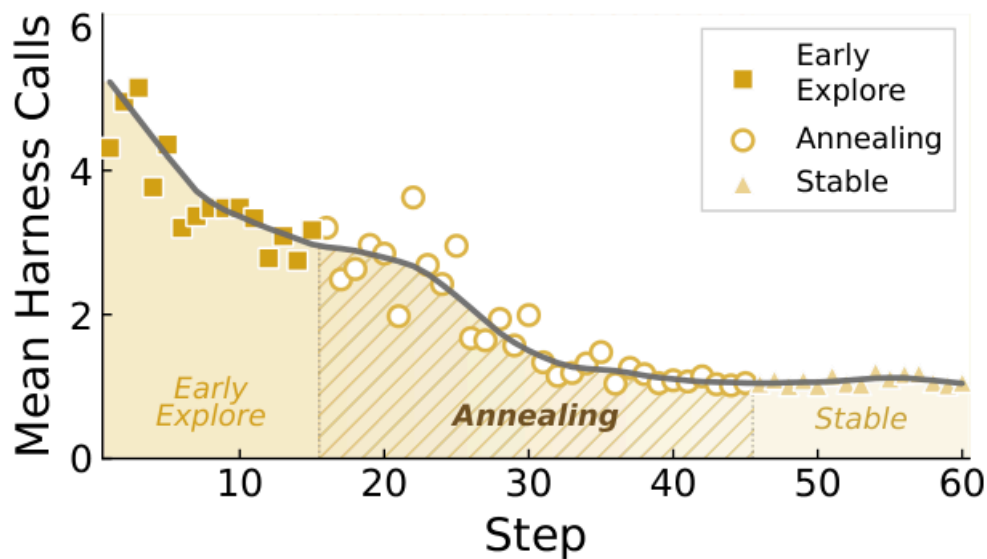


그림 5-1 GRPO 진행에 따른 하니스 호출 수 — 잦은 인지 도구 호출에서 에피소드당 근접 1회로 안정

그림 5-1의 곡선은 단순하지만 뜻은 깊다. SFT로 초기화한 에이전트는 훈련 초기에 하니스를 잦게 부른다. 상태를 추적하고, 절차를 꺼내 보고, 탐색 공간을 좁히는 수단으로 BPE(Belief-Progress-Experience)를 걸으며 드러나는 비계 삼아 쓰는 것이다. GRPO가 진행되면서 호출 수는 빠르게 떨어지고, 이내 **에피소드당 근접 한 번**의 수준에서 안정된다.

담금질이라는 이름은 이 책의 풀이로는 이렇다. 뜨겁게 시작한 호출이 학습과 함께 식어 가며 안정점을 찾는 모양이, 열을 가했다 식히며 성질을 굳히는 풀과 닮았다. 논문 자신의 해석은 더 담백하다. GRPO가 되풀이되는 비계 행동을 정책 안으로 점차 내재화하되, 기대하는 이득이 단계 비용을 웃도는 경우에만 하니스 접근을 남겨 둔다는 것이다. 무엇이든 밖에 적어 두고 움직이던 에이전트가, 배울수록 꼭 필요한 순간에만 밖을 들여다보게 된다. 논문의 표현을 빌리면 학습은 비계에 기댄 탐색에서 선별적이고 비용을 헤아리는 조정으로 옮겨 간다.

하니스 진화 — 팽창과 통합을 거치는 기술 은행

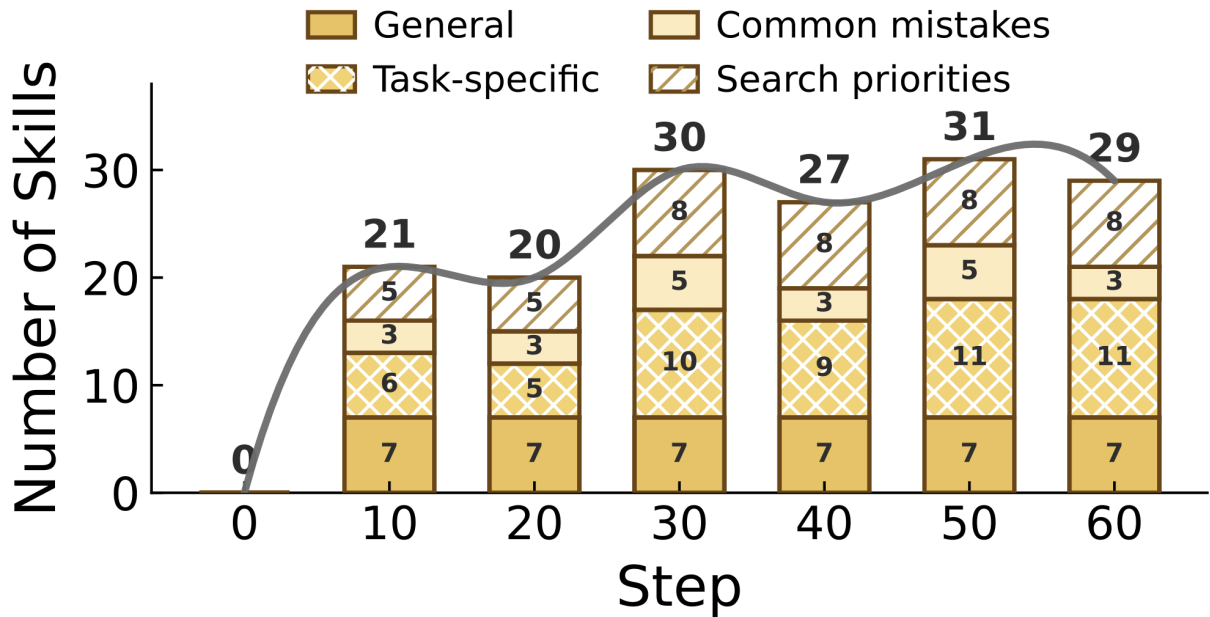


그림 5-2 경험 저장소 진화 — 기술 은행이 탐색 초반에 팽창했다가 콤팩트한 혼합으로 안정

정책이 하니스를 덜 부르는 쪽으로 움직이는 동안 하니스 자체는 가만히 있지 않다. BPE의 세 구성 요소 가운데 신뢰 상태 (belief)와 진행도(progress)는 에피소드 안에서 갱신되지만, 경험(experience)은 에피소드를 넘어 축적과 통합과 망각을 거치며 재편된다. 그림 5-2가 보여 주는 것이 바로 이 경험 저장소의 궤적이다.

기술 은행(skill bank)은 훈련 초반에 빠르게 팽창한다. 일반 전략과 과제별 절차, 흔히 저지르는 실수, 탐색의 우선순위가 저마다 항목으로 쌓인다. 후반으로 가면 성장은 선별적으로 바뀐다. 중복 항목은 하나로 병합되고, 드물게 쓰이는 기술은 밀려나고, 자주 회상되는 지식은 남는다. 최종 은행은 크지 않으면서도 다양하다. 논문은 이 모습을 두고, 하니스가 무작정 쌓기만 하는 메모리가 아니라 과제에 맞춰 적응하는 상태 기판이 되었다고 요약한다.

담금질과 진화는 서로를 보완하는 두 움직임이다. 정책은 **언제** 외부 상태를 쓸지 배우고, 하니스는 **무엇을** 재사용 가능한 경험으로 내놓을지 바꾼다. 한쪽 그림만 보면 이 학습의 전체 모양새가 보이지 않는 이유다.

액션별 담금질 — 네 액션은 같은 속도로 사라지지 않는다

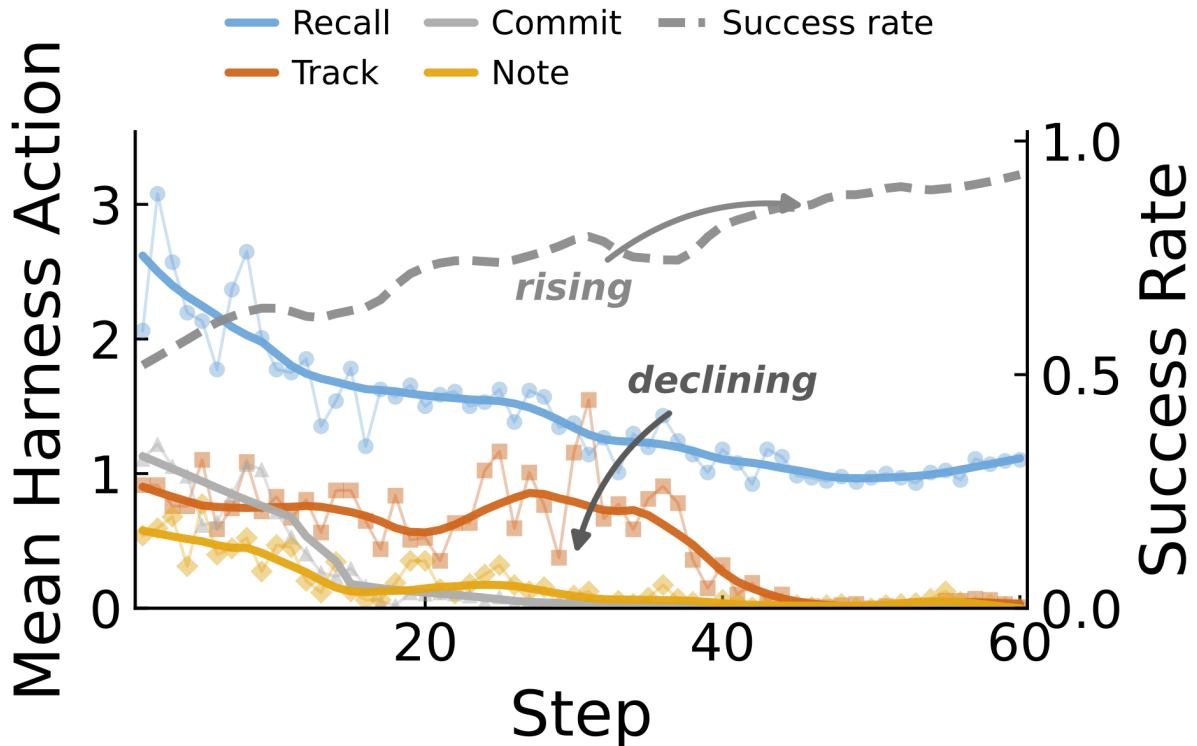


그림 5-3 액션별 하니스 담금질 — BPE 액션마다 감소 속도가 다르지만 성공률은 오른다

부록 A는 호출 수의 총량을 액션 유형별로 쪼개 본다. 그림 5-3이 보여 주듯 네 액션은 균일하게 가지치기되지 않는다. 초기 에이전트가 경험 조화에 더해 추적(track)과 진행 확정(commit), 메모(note)를 두루 부른 것은 SFT 비계의 흔적과 같다. GRPO가 진행되면 이 가운데 무엇을 남길지가 갈린다.

회상(recall)은 끝까지 가장 잘 남는 액션이다. 루틴적인 행동이 상당수 내재화된 뒤에도 에피소드를 넘는 경험이 여전히 유용한 탐색 사전 지식을 준다는 뜻이다. 반면 확정과 메모는 빠르게 0을 향해 줄어든다. 안정된 과제 전략이 자리 잡고 나면 중간 계획을 일일이 밖에 적거나 잦은 새 통찰을 기록할 필요가 없어진다. 추적은 그 중간에 있다. 초기에는 상태를 가려 내는 데 요긴하지만, 에이전트가 환경과 더 직접 상호작용하는 방식을 익히면서 점차 줄어든다.

눈여겨볼 대목은 이 감소 곡선들과 성공률의 방향이 어긋난다는 점이다. 호출은 줄어드는데 성공률은 계속 오른다. 논문은 이 액션별 무늬가 환경에 따라 달라진다는 점도 분명히 한다. ALFWorld는 집안일 절차와 물건 탐색의 사전 지식이 과제 사이에서 공유되는 터라 경험 구성 요소가 유독 값진 환경이다. 논문의 추정을 따르면 장면 그래프와 객체 상태, 공간 관계를 붙들어야 하는 시각 중심의 실물 환경에서는 신뢰 상태에, 하위 과제와 테스트 상태, 미완료 분기를 좇아야 하는 소프트웨어 엔지니어링 환경에서는 진행도에 무게가 실릴 수 있다. 요컨대 EvoHarness-RL이 배우는 것은 어디서나 통하는 고정된 액션 배분이 아니라, 그 환경의 비용 구조와 상태 요구에 맞춰 BPE의 어느 부분이 접근할 값어치가 있는지도.

보상 궤적 — 덜 부르는 것과 못 쓰는 것은 다르다

호출 수가 줄어드는 그래프를 처음 보면 정책이 하니스를 잃어버린 건 아닌지 의심이 먼저 든다. 부록 A의 보상 비교가 그 의심을 잠재운다. 훈련을 통틀어 EvoHarness-RL의 보상은 표준 GRPO를 일관되게 앞선다. 오르는 속도가 더 빠르고 높은 평형도 더 높으며, 표준 GRPO는 개선이 느려 상당히 낮은 자리에 머문다.

논문은 이 결과를 액션별 담금질과 나란히 읽는다. 하니스 호출의 감소는 정책이 무너졌거나 하니스를 쓰지 못해서 생긴 것이 아니다. BPE가 먼저 탐색과 상태 관리의 비계로 기능하고, GRPO가 되풀이되는 비계 행동을 안으로 걷어 들이되 가장

유용한 외부 상태에 대한 선별적 접근은 남겨 두는, 바로 그런 학습의 결과라는 것이다. 덜 부르는 것과 못 쓰는 것을 갈라 보는 이 비교가 없었다면 담금질은 오해의 소지가 있는 그래프로 남았을 것이다.

이 장의 해석 — 외주에서 내재화로

이 책은 이 장의 두 그림을 한 문장으로 줄인다. 학습은 하니스를 떼어 내는 과정이 아니라 하니스를 향한 손짓의 수를 줄이는 과정이다. 되풀이되는 일은 정책이 안으로 삼키고, 에피소드를 넘어서까지 값을 매길 수 있는 경험만 밖에 남는다. 회상이 끝까지 남는다는 관찰이 여기서 힘을 얻는다. 에피소드당 근접 한 번의 호출이 마지막까지 남은 그 몫을 채우는 셈이다.

여기에 하니스 쪽 움직임은 더하면 그림이 완성된다. 정책이 접근을 줄이는 동안 기술 은행은 통합과 망각으로 스스로 다듬어져, 정책이 찾을 가치가 있는 경험만 좁혀 둔다. 한쪽은 부르는 횟수를, 다른 쪽은 내놓는 내용을 줄인다. 방향은 다르지만 둘 다 비용을 헤아리는 같은 학습의 두 측면이며, 이 두 겹이 맞물릴 때 하니스는 사람이 정해 준 고정 구조가 아니라 학습되어 바뀌는 무엇이 된다.

물론 위의 요약은 이 책의 읽음이지 논문의 문장은 아니다. 논문이 보여 준 것은 ALFWorld라는 한 환경에서의 그래프이고, 액션별 무늬가 환경을 바꾸면 어떻게 달라질지는 앞서 본 추정의 자리에 머문다. 담금질과 진화가 다른 환경에서도 같은 모양으로 나타나는지는 열린 질문으로 두는 것이 이 논문을 정확히 읽는 태도다.

DAY

06

제6장 — 관련 연구와 시리즈의 자리



“원문: EvoHarness-RL — arXiv:2608.05446 · §5 Related Work, §6 Conclusion, Appendix B·D”

이 장이 읽는 것은 원문 §5(관련 연구), §6(결론), 그리고 두 부록이다. 부록 B는 학습된 정책이 실제 궤적에서 하니스와 조율하는 정성 사례이고, 부록 D는 그 조율을 떠받치는 프롬프트 전문이다. 본문의 논증이 끝난 자리에서 이 연구의 흐름과 이 책의 시리즈 안 자리를 정리한다.

■ 관련 연구의 두 갈래

원문 §5는 선행 연구를 두 갈래로 나눈다. 첫 갈래는 하니스 엔지니어링이다. 초기 연구가 명시적 추론·행동 루프(ReAct)와 기본적인 도구 증강을 제시했다면, 이후 연구는 관측 렌더러, 파일 시스템 접근, 실행 피드백을 갖춘 정교한 실행 환경으로 발전했다. 이 구성 요소들이 장기 과제 성공을 좌우하면서 최근 연구는 하니스 자체를 최적화한다. Harness-1은 검색 상태를 환경 쪽 메모리로 끌어내 외재화하고, Meta-Harness와 HarnessX는 오프라인 검색과 궤적 기반 적응으로 효과적인 하니스 구성을 찾아낸다. 원문이 짚는 공통 한계는 이것이다. 이들은 하니스를 환경 쪽 구조물, 또는 사람 개발자나 오프라인 탐색 알고리즘이 다듬는 프롬프트 시점의 관례로 취급한다.

둘째 갈래는 메모리와 자기진화 에이전트다. 초기 시스템들은 궤적과 반성, 과제 요약을 쌓아 다음 에피소드에 활용했다(Reflexion). 원시 로그가 시끄럽다는 점이 드러나자 후속 연구는 경험을 워크플로, 코드 라이브러리, 재사용 스킬 같은 구조화된 절차 기억으로 증류한다(ExpeL, Voyager, MemP, ReasoningBank). 최근 연구는 경험 저장소를 쌓기만 하는 로그가 아니라 능동적으로 다듬고 통합하고 잊어야 할 대상으로 본다(SkillIOS, SkillRL). 이 갈래의 공통 한계도 원문이 분명하게 적는다. 에피소드를 걸친 스킬 큐레이션이 실시간 상태 추적 — 환경에 대한 믿음 유지와 서브골 진행 감시 — 와 분리되어 있다는 것이다.

■ 장비를 다듬는 연구, 사용법을 학습하는 연구

두 갈래를 나란히 놓으면 이 논문의 자리가 선명해진다. 두 갈래가 모두 하니스와 메모리라는 **장비**를 다듬는다면, EvoHarness-RL은 그 장비를 다루는 **사용 정책**을 학습한다. 하니스 접근을 학습 가능한 정책 결정의 자리에 올려놓고, 더 나은 고정 스캐폴드를 얹어 주는 대신 정책 스스로 외부 작업 공간을 통제하고 조화하고 조율하도록 훈련한다. 절차 기억 갈래의 한계에는 BPE 추상화로 답한다. 믿음·진척·경험을 한 작업 공간에 두면 에이전트는 장기 경험을 쓰는 방식만이 아니라, 그 경험을 현재의 믿음과 진척에 맞추는 방식까지 익히게 된다. 원문 결론의 한 문장이 이를 압축한다. 하니스 사용을 수동적인 프롬프팅 관례에서 학습된 런타임 정책으로 바꾼다.

■ 서른 번의 턴에 담긴 조율 — 부록 B

부록 B는 ALFWorld의 한 궤적을 통째로 보여 준다. 과제는 주전자를 씻어 식탁에 놓는 것이고, 서른 번의 턴 만에 성공한다. 스킬 뱅크에는 단계 순서 계획(찾기 → 씻기 → 놓기)과 물건 위치 사전이 들어 있고, 피할 목록에는 낡은 힌트 맵신과 같은 장소 반복 탐색이 적혀 있다.

궤적은 commit → recall → act → note의 한 바퀴로 흘러간다. 에이전트는 행동에 앞서 첫 서브골을 등록한다. 이어 과제 수행법과 주전자 위치를 물으니 돌아온 힌트는 카운터탑을 가리켰다. 그런데 카운터탑 둘 다 비어 있다. 힌트가 두 번 빗나가자 같은 장소를 되풀이하지 말라는 경고를 따라 루프를 끊고, 주전자는 흔히 스토브버너에 있다는 믿음으로 탐색을 옮긴다. 스토브버너 셋에서 주전자를 찾는 순간 낡은 기억과 현재의 관측이 엇갈렸음이 확인된다. 에이전트는 기억을 고치는 쪽에 선다. 주전자를 집은 뒤 note로 스토브버너 셋에서 찾았고 힌트는 카운터탑이라 했다는 기록을 남긴다. 마지막으로 다음 서브골을 등록하고 싱크대에서 씻어 식탁에 놓으며 과제를 닫는다.

원문이 뽑는 요점은 이것이다. 하니스는 고정된 신탁이 아니라 **재사용 가능하되 수정 가능한 경험**의 원천이며, 에이전트는 에피소드를 걸친 사전 지식과 현재 에피소드의 확인·자기수정을 한 궤적 안에서 결합한다. 앞 장의 숫자와 곡선이 말했던 조율이 여기서는 한 바퀴의 루프로 눈에 보인다.

프롬프트는 문서가 아니라 규격이다 — 부록 D

부록 D는 두 가족의 프롬프트를 심는다. 정책이 매 스텝 보는 런타임 하니스 프롬프트와, 요약 모델이 에폭 경계에서 쓰는 경험 저장소 통합 프롬프트다. 전문을 옮기는 대신 뼈대를 읽는다.

런타임 프롬프트는 네 겹으로 짜여 있다. 첫째 환경 액션 — 허용 명령 목록에 있는 것만 정확히 고른다. 둘째 하니스 액션 — 계획의 commit, 시각의 track, 경험의 recall·note 네 가지를 인지 도구로 규정하고, 언제나 쓸 수 있되 각각 한 스텝의 값을 치른다고 밝힌다. 셋째 사용 시점 — commit은 과제 시작과 서브골 전환 시, recall은 위치·절차·실수 세 가지 질의로 에피소드 내내, track은 봤지만 만지지 않은 물건을 되살릴 때, note는 정해진 세 조건에서 의무로. note의 의무 조건이 눈 길을 끈다. 힌트가 비어 있었거나, 힌트와 다른 곳에서 물건을 찾았거나, 절차 질의가 비어 있었을 때 기록을 남겨야 한다. 부록 B의 수정 note는 바로 이 조건에서 나왔다. 넷째 출력 형식 — 생각 한두 문장과 액션 하나를 정해진 꼴로 내놓는다.

통합 프롬프트는 저장소의 문을 지킨다. 정책이 쓴 note는 공장 저장소에 들어가지 않고 버퍼에 쌓인다. 통합 시점에 요약 모델이 최근 note와 기존 스킬 बैं크를 함께 받아 노트마다 add·update·remove·skip 넷 중 하나를 판정한다. 겹치면 갱신을 추가보다 우선하고 삭제는 명백한 무효화에만 쓰라는 규칙이 붙어 있다. 이 길을 지나 정책의 증거가 중복 없고 스스로 고쳐지는 스킬이 된다. 나머지 네 개의 배치 유도 프롬프트는 일반 스킬, 과제 특화 스킬, 흔한 실수, 검색 우선순위 범주를 처음에 뿌리고 주기적으로 다시 쌓는다.

원문이 강조하는 설계 판단 하나를 남긴다. SFT 교사와 GRPO 정책이 같은 런타임 프롬프트를 보고, 두 시점의 스킬 통합이 같은 저장소 인터페이스를 지난다. 두 학습 단계가 구성 자체에서 프롬프트 일치한다는 것이다. 내 읽기로 여기서 프롬프트는 사람을 위한 문서가 아니라 두 학습 단계가 같은 세계에서 만나게 하는 규격이다.

시리즈의 세 점 — 안, 바깥, 그리고 사이

이 대목부터는 논문 밖의 이야기, 시리즈의 언어다. 『에이전트는 대화로 일하지 않는다』는 SKILL.state를 읽으며 하니스 안쪽을 다뤘다. 실행 상태를 호출 안의 컨텍스트에서 꺼내 반복 사이에 사는 기록으로 옮기는 일, 컨텍스트는 호출 안에서만 살고 기록은 반복 사이를 산다는 교훈이 그 책의 한복판이었다. 『하니스의 하니스』는 바깥쪽을 다뤘다. 하니스 자체를 유지보수하는 순환, 즉 산출물 계보와 증거번들을 남기며 다음 반복이 상속받을 것을 지금 상태에 적어 두는 관행이다. 그리고 이 논문은 사이를 다룬다. 안쪽의 상태를 지금 읽을지 쓸지 무시할지를 에이전트가 학습하는 문제, 둘을 잇는 관절 그 자체를 정책으로 만든다.

세 권을 나란히 놓으면 질문이 세 개로 갈라진다. 상태는 어디에 사는가, 상태는 누가 돌보는가, 상태를 향한 행동은 누가 정하는가. SKILL.state가 기록의 스키마를, HoH가 기록을 검증하며 돌보는 순환을, EvoHarness-RL이 기록을 향한 조율을 각각 담당받는다. 내 생각에 이 세 질문은 하나의 시스템 안에서 동시에 성립해야 온전하다. 기록이 없으면 조율할 대상이 없고, 돌보는 순환이 없으면 기록은 쌓일 뿐이며, 조율이 사람의 관례에 머물면 하니스가 아무리 풍성해져도 사용법은 제 자리에 박힌다.

남는 질문

논문이 스스로 남겨 둔 지점이 있다. 먼저 이식성이다. 원문은 네 메타 액션으로 인터페이스를 좁힌 이유를 도메인 특화 API는 학습된 행동을 옮기기 어렵게 만든다는 데 두었다. 어댑터는 공유를 전제로 설계된 셈이다. 그러나 부록 A는 학습된 액션 분포가 환경 의존적임도 밝힌다. ALFWorld에서는 경험이 무겁고, 시각 환경에서는 믿음, 소프트웨어 환경에서는 진척이 무거워지리라는 것이다. 고정된 보편 분포가 아니라 주어진 비용 구조와 상태 요구에 맞춰 어디에 접근할 가치가 있는지 배운다. 인터페이스는 옮겨도 정책은 그 환경의 비용 구조에 보정된 값이라는 뜻이고, 미지 과제 86.6% 역시 ALFWorld 안의 일반화다. §3.4의 관찰도 같은 맥락이다. 모방 학습 변형은 미지 분할에서 69.4%로 프롬프트 시점 하니스의 77.6%보다 아래로 떨어지는데, 원문은 교사의 하니스 사용 패턴을 본뜬 것만 접근이 값진 때를 최적화하지 못한다는 해석을 직접 붙인다. 다음 환경으로 넘어가는 것은 무엇인가 — 인터페이스인가 정책인가.

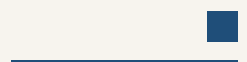
비용 모델도 같은 방향의 질문이다. 이 논문에서 하니스 액션 한 번은 한 스텝과 같은 예산이고, 학습된 절제는 그 특정 비용 구조 안의 절제다. 스텝이 아니라 토큰이나 지연 시간, 금전 비용으로 값을 매기는 환경에서 같은 정책이 그대로 옮겨지

는 열려 있다. 원문의 표현을 빌리면 학습은 언제나 주어진 환경의 비용 구조 안에서 일어난다. 무엇을 공유하고 무엇을 다시 배워야 하는가 — 어댑터와 정책을 갈라 놓은 이 설계가 다음 연구에게 남긴 질문이라고 읽는다.

DAY

07

맺음말 — 우리 시스템에 남는 것



논문을 한 문장으로 압축하면 이렇다. **하니스가 에이전트를 지탱하듯, 학습된 정책이 하니스의 사용을 지탱한다.** 장비를 더 없는 대신, 언제 믿음을 읽고 진척을 적고 경험을 꺼낼지를 정책 안으로 끌어들여 ALFWorld에서 96.9%까지 올렸다. 이 결론을 우리 시스템에 비추면 남는 것이 몇 개 있다.

하나 — 상태는 역할로 나눈다

BPE의 첫 설계 판단은 저장소를 늘리는 것이 아니라 역할을 좁히는 것이었다. 이질적인 외부 장비를 믿음·진척·경험 세 역할로 추상화하자 8B 모델도 학습할 수 있는 좁은 인터페이스가 남았다. 우리의 SKILL.state 적용과 정확히 같은 방향이다. 실행 상태를 스키마로 좁혀 두면 재개는 유추가 아니라 상태 읽기가 된다. 상태 설계에서 물을 질문도 그대로 쓴다. 이 항목은 **환경에 대한 믿음인가, 실행의 진척인가, 겹쳐온 경험인가** — 셋 중 어느 것도 아니라면 상태가 아니라 로그다.

둘 — 읽기에도 비용을 매긴다

이 논문의 담금질 관찰이 찌르는 지점은 명료하다. 하니스 액션은 환경 액션과 같은 예산을 태운다. 그래서 학습된 에이전트는 호출을 줄여 간다. 우리 관행에 비추면 이것은 기록 열람에도 같은 질문을 던진다. **지금 이걸 다시 읽는 것이 한 걸음 진행보다 값진가.** 재검색·재열람·재검토를 습관으로 두는 시스템은, 이 논문의 표현으로 말하면, 담금질되지 않은 정책이다. effort-router의 라운드 상한이 지키려는 것도 결국 같은 예산 감각이다.

셋 — 되풀이되는 사용법은 내재화한다

담금질의 본질은 외주를 안으로 가져오는 것이다. 자주 반복되는 track-commit 패턴은 GRPO를 거치며 정책의 무게로 굳는다. 우리 시스템에서 이 일은 스킬과 에이전트 정의가 맡는다. 세션마다 되풀이되는 절차를 매번 대화로 다시 짜는 대신, 한 번 스킬로 굳혀 두는 것이 곧 내재화다. 그리고 내재화의 조건도 논문이 알려 준다 — 감독 학습이 먼저다. 쓰임새를 익힌 뒤에 쓸 때를 익힌다. 절차를 스킬로 굳히기 전에 그 절차가 실제 궤적에서 되풀이됐는지부터 확인하는 것, 그것이 우리의 SFT 단계다.

넷 — 경험은 쌓는 것이 아니라 다듬는 것이다

경험 저장소의 곡선은 인상적이다. 탐색기에 팽창했다가 통합을 거치며 콤팩트한 혼합으로 수렴한다. note로 아무거나 쌓기만 하면 창고는 커지고, add-update-remove의 통합이 있어야 과제에 맞는 기질로 좁혀진다. 우리의 후보 등록 전 source_refs 대조 규칙이 같은 자리다. 기억 후보가 들어올 때 기존 항목과 겹치는지 대조해 흡수할지 버릴지 정하는 관행이 곧 에포크 경계의 통합 모델이다. 쌓는 시스템은 결국 recall을 망친다.

다섯 — 장비의 시대 다음은 사용 정책의 시대이다

시리즈 세 권을 나란히 놓으면 결론이 하나로 겹친다. SKILL.state는 그릇을 대화에서 상태로 옮겼고, HoH는 하니스의 유지보수를 사람에서 바깥 루프로 넘겼고, 이 논문은 하니스를 쓰는 법 자체를 프롬프트에서 정책으로 가져왔다. 세 권 모두 같은 진단에서 출발한다 — **능력의 병목은 모델 안에 아니라 모델을 둘러싼 운영 틀에 있다.** 더 강한 도구를 엮기 전에, 그 도구를 언제 만질지를 먼저 설계하는 것. 그 순서를 뒤집지 않는 것. 이 책이 우리 시스템에 남기는 것도 결국 한 줄이다. **하니스를 더 엮기 전에, 지금 그 하니스를 읽어야 하는지 먼저 묻는다.**